

Lecture 15

Deep Learning (Agent Perspective)

COMP532 – Machine Learning and Bioinspired Optimisation

Meng Fang, University of Liverpool



What Is an LLM?

A Large Language Model is:

- ▶ A transformer-based neural network
- ▶ Trained on large-scale text data
- ▶ Optimized for next-token prediction

Next-Token Prediction

LLMs optimize:

$$p(x_{t+1} \mid x_{\leq t})$$

Joint factorization:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

Example

Context:

“The capital of France is”

Model predicts:

- ▶ Paris: 0.82
- ▶ London: 0.05
- ▶ Berlin: 0.04

Training: Maximum Likelihood

Objective:

$$\mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t})$$

Equivalent to minimizing cross-entropy loss.

Teacher Forcing

- ▶ Training: use ground-truth history
- ▶ Inference: use model-generated history

This creates exposure bias.

Autoregressive Generation

Given prompt:

$$x_1, \dots, x_k$$

Generate sequentially:

$$x_{k+1}, x_{k+2}, \dots$$

Decoding methods:

- ▶ Greedy
- ▶ Sampling (Top-k, Top-p)
- ▶ Beam search

Remark

From this simple objective emerge: reasoning, translation, coding, planning.

Pretraining

Pretraining stage:

- ▶ Massive unlabeled text
- ▶ Self-supervised learning (next-token prediction)

Example training sequence

“The cats are playing in the garden”

Model must predict:

“are” after “cats”

Model gradually learns:

- ▶ Grammar (subject–verb agreement)
- ▶ Semantics (cats are animate)
- ▶ World patterns (animals play, gardens exist)

Fine-Tuning

After pretraining, we optimize:

$$\mathcal{L} = - \sum \log p_{\theta}(y \mid x)$$

where:

- ▶ x = instruction
- ▶ y = desired response

Example

Input:

“Translate to French: Hello”

Target:

“Bonjour”

Alignment: Why Do We Need It?

After SFT, the model can follow instructions.

But it may still generate:

- ▶ Unsafe content
- ▶ Biased responses
- ▶ Unhelpful answers

Example

Prompt:

“How can I hack into someone’s account?”

Pretrained/SFT model:

Provides technical instructions

Aligned model:

Refuses and explains safety concerns

Goal: Make responses helpful and safe.

Alignment as Objective Change

Pretraining:

$$\max \log p(x_t \mid x_{<t})$$

SFT:

$$\max \log p(y \mid x)$$

Alignment (RLHF / DPO):

$$\max_{\theta} \mathbb{E}_{y \sim p_{\theta}(\cdot|x)} [r(x, y)]$$

Key idea: Change the objective, not the architecture.

Retrieval-Augmented Generation (RAG)

RAG pipeline:

- ▶ Retrieve relevant documents d
- ▶ Feed them into the prompt
- ▶ Generate answer

Standard LLM:

$$p(y \mid x)$$

RAG:

$$p(y \mid x, d)$$

where d = retrieved documents.

RAG adds external memory to the model.

Example

Question: “What is the acceptance rate of ACL 2023?”

Retrieved document snippet:

“ACL 2023 acceptance rate was 23.4%.”

Model generates answer using retrieved evidence.

Why RAG?

LLMs:

- ▶ Have fixed internal knowledge
- ▶ May hallucinate

Example

“Who won the 2025 Nobel Prize in Physics?”

A model trained in 2023 cannot know the answer.

RAG:

- ▶ Injects up-to-date information
- ▶ Improves factual accuracy

Tool Use

LLMs can call external tools:

- ▶ Search engines
- ▶ Calculators
- ▶ Code interpreters

Example

User: What is 17×23 ?

Model output:

```
{  
  "tool": "calculator",  
  "arguments": {"expression": "17*23"}  
}
```

Tool returns: 391

LLM as an Agent

Agent loop:

- ▶ Observe context
- ▶ Plan
- ▶ Act (possibly via tools)
- ▶ Update context

Example: Book a flight

- ▶ Observe: User wants a flight to Paris next Monday
- ▶ Plan: Search flights → Compare prices → Select option
- ▶ Act: Call flight search API
- ▶ Update: Present best option to user

Memory in LLM Systems

Types of memory:

- ▶ Context window
- ▶ External vector database
- ▶ Structured memory store

Example: Long conversation

- ▶ Context window: remembers last 8k–128k tokens only
- ▶ Vector DB: retrieves relevant past discussion
- ▶ Structured store: saves explicit user facts (e.g., name, preferences)

Limitations of LLMs

LLMs may:

- ▶ Hallucinate e.g., fabricate non-existent papers or citations
- ▶ Struggle with precise reasoning e.g., make arithmetic or logical errors
- ▶ Reflect biases in data e.g., generate stereotypical associations

Key reason: The objective optimizes likelihood, not truth.

Key Insight: From LLM to Agent

LLMs are:

- ▶ Powerful pattern predictors
- ▶ Not explicit reasoning engines

Agents require:

- ▶ Iterative decision-making
- ▶ Memory and state
- ▶ Tool interaction
- ▶ Feedback and evaluation

Insight:

An agent is not just a model — it is a model embedded in a control loop.

Key Insight: From LLM to Agent

LLM:

$$y \sim p_{\theta}(y \mid x)$$

Agent:

$$a_t \sim \pi_{\theta}(a_t \mid s_t)$$

$$s_{t+1} = f(s_t, a_t)$$

Insight:

LLM = one-step prediction

Agent = sequential decision-making with state

Big Picture

Neural networks

Function approximation

Transformers

Sequence modeling with attention

LLMs

Large-scale probabilistic language models

AI Systems

Models + Memory + Tools + Control loops

Insight: Architecture + Data + System integration

Key Takeaways

- ▶ LLMs are probabilistic sequence models.
- ▶ Objectives shape behaviour (pretraining → alignment).
- ▶ External memory and tools extend capability.
- ▶ Agents embed models into decision loops.

Final insight:

Modern AI = Model + Memory + Tools + Control.